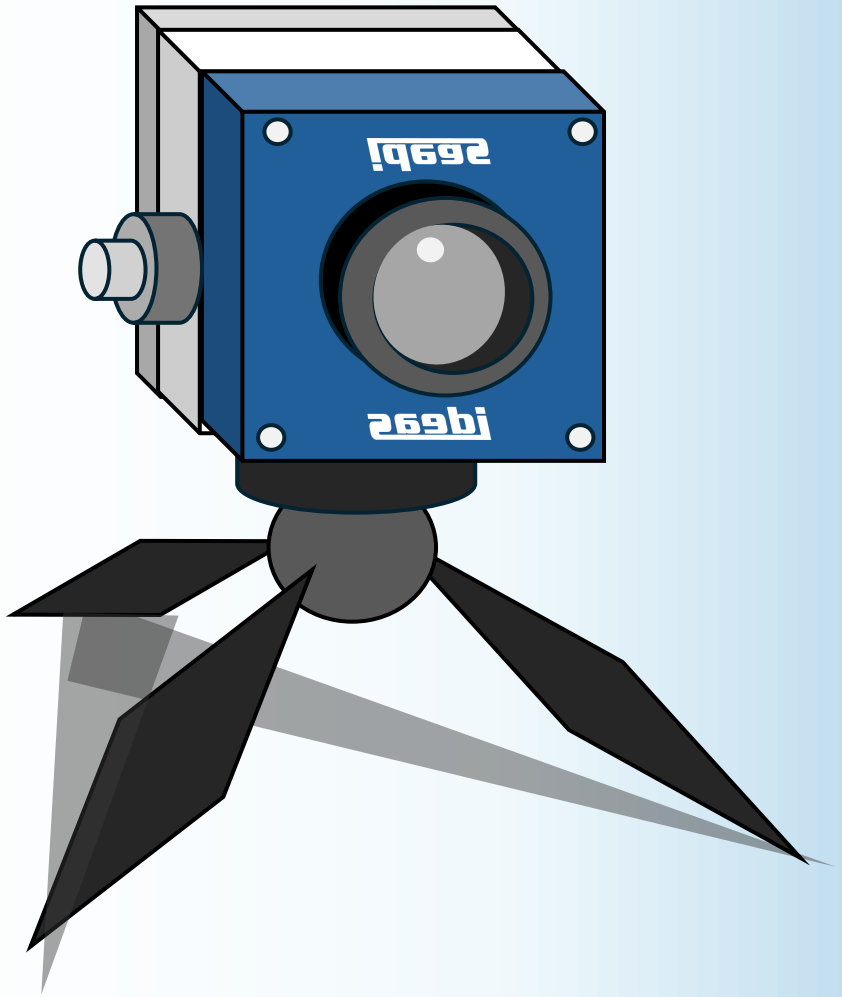




Real-time control software for a thermal imaging system at Integrated Detector Electronics AS (IDEAS)

Vu Hoang Nguyen 2026

Tasks overview

Goal

- Build a functional desktop application to visualise the thermal video in real time
- Provide the measurement, calibration and recording functions needed for camera characterisation

Overview of the thermal camera system

Mobile System for Demonstrations

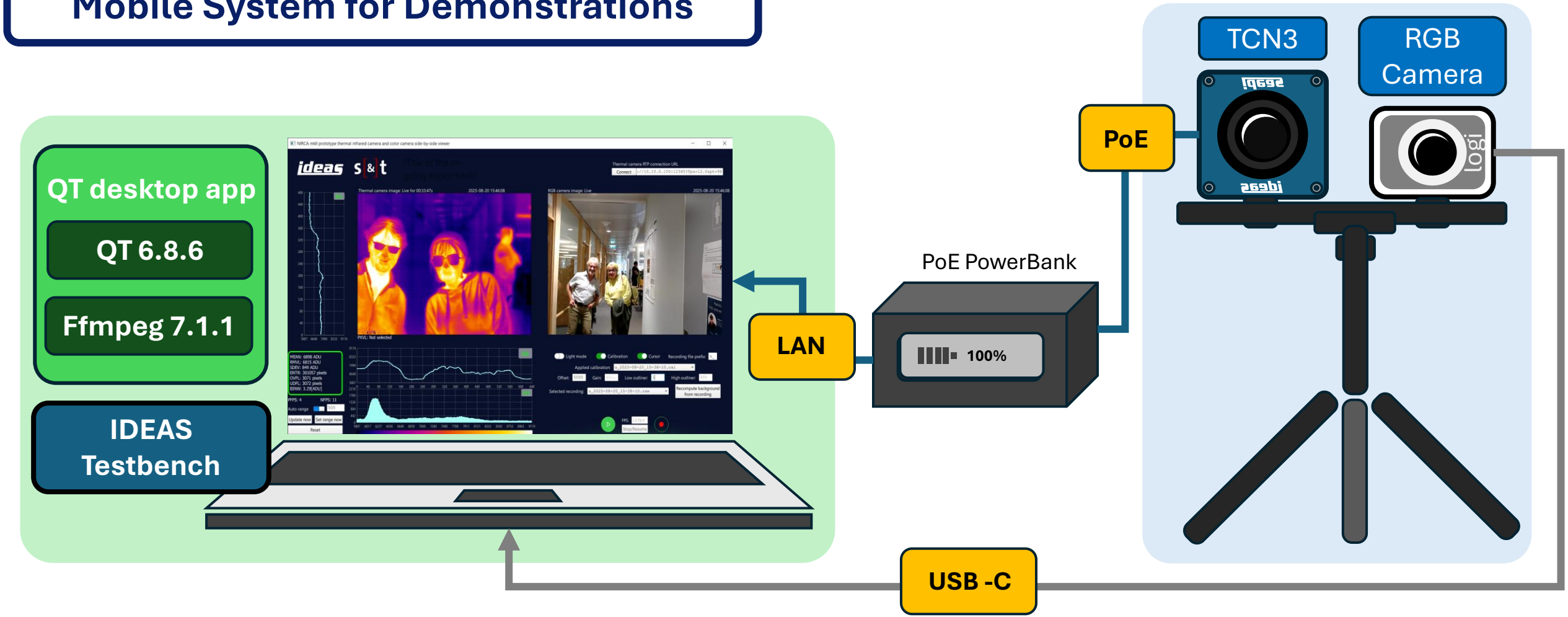
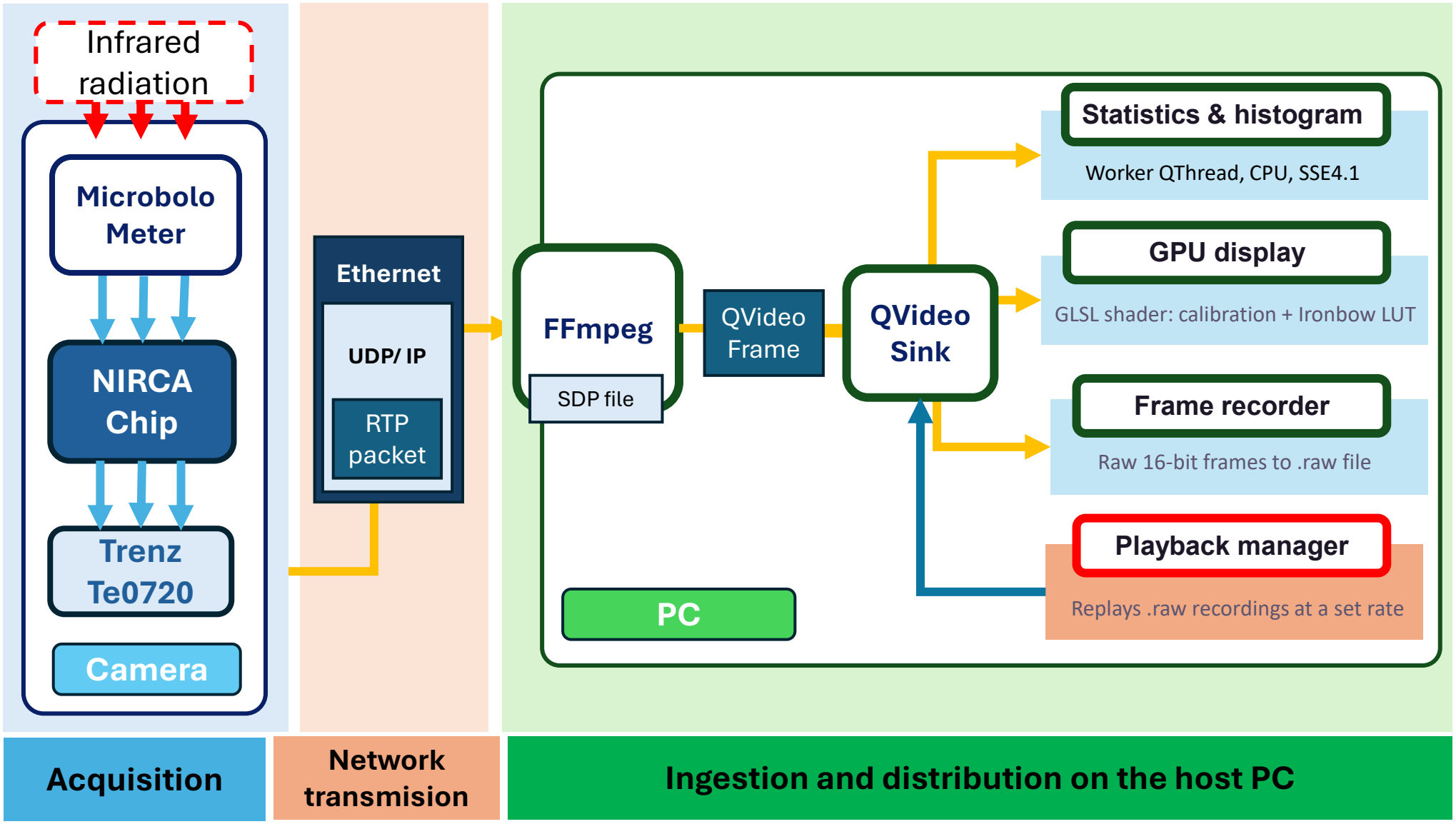


Figure 4: Thermal camera mobole system at IDEAS

Overview of the data flow



Acquisition
Infrared radiation warms each microbolometer pixel; the NIRCA chip digitises the array to 14 bits, and the TE0720 module packs the frames for the network.

Network transmission
One frame is 614 kB, far above the Ethernet MTU, so it is split across many RTP packets over UDP: low latency, with sequence numbers to reassemble it.

Ingestion and distribution
An SDP file generated at runtime tells FFmpeg to expect raw 16-bit greyscale. Every consumer then draws from one QVideoSink: display, statistics, recording, replay.

Layout of user interface

Software GUI

Thermal and RGB camera

Set connection

Graphs

Statistic

Histogram

Calibration and record



IDEAS Company Confidential Information

Core functions of the software

Graphs and histogram

A 1024-bin histogram of pixel values, switchable between a linear and a logarithmic Y axis to bring out the tails, plus row and column average plots. All four modes are drawn by one custom QPainter component.

Display range

Auto-range recomputes the limits every few seconds, discarding the outer 1% of pixels at each end so dead and hot pixels cannot flatten the contrast. Both the percentage and the update period are set from the interface. The range can also be typed in manually, or frozen from the current auto-range values with one button.

Statistics

Mean, running mean and standard deviation over in-range pixels, with overflow and underflow counts and separate received and processed frame rates.

Crosshair

Select a pixel and read its position and calibrated value, reported on every frame with an underflow or overflow marker. The row and column graphs switch from averages to cross-section profiles through that point.

Record and replay

Raw 16-bit frames are written to file and replayed at an adjustable rate through the same video sink as live frames, so every downstream stage behaves identically.

Display and layout

The thermal image with Ironbow colour mapping beside an RGB reference view, plus dark and light themes for lab and outdoor use.

Chart rendering

Choosing a chart renderer that stays stable

ATTEMPT 1: QTGraph 2D

Started with QtCharts and then migrated the Qt Charts code following the official guidelines. Construction errors instead of plots; Qt 6.8 was the module's first stable release.

Abandoned

ATTEMPT 2: QTGraph 3D

Rendered bars in 3D with the camera locked onto the XY plane. It worked, but perspective and shading artefacts made the plot uncomfortable to read.

Abandoned

ATTEMPT 3:custom QQuickPaintedItem

One QPainter-based class, four modes selected from QML, with slots for data, range and colour. Stable and readable at the camera rate.

Adopted

Platform migration

Upgraded Qt 6.7.3 to the 6.8.3 LTS release with FFmpeg 7.1.1. Even this step touched declarations, type names and imports across the code base, so a direct jump to Qt 6.10 was judged too risky to debug. (QtGraph in 6.10 is expected to be more mature and stable)

Trade-off accepted

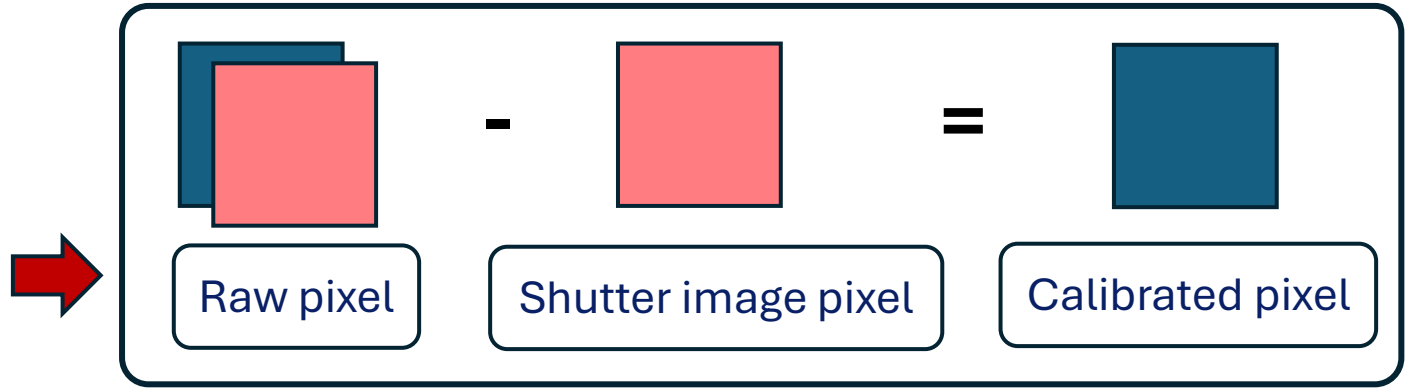
CPU painting gives up GPU acceleration. That was acceptable: the charts render without flicker at the camera rate, and reliability and readable styling mattered more for daily lab use.

Calibration method for non-uniformity

I 1 point Shutter correction

Basic principle

Using a shutter image as a reference



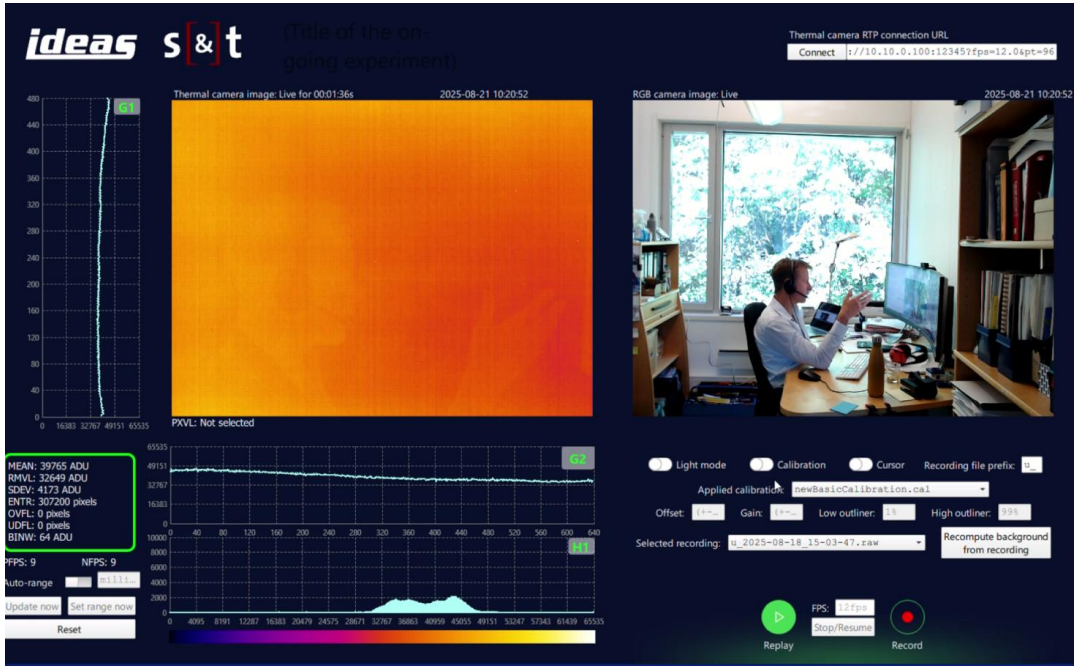
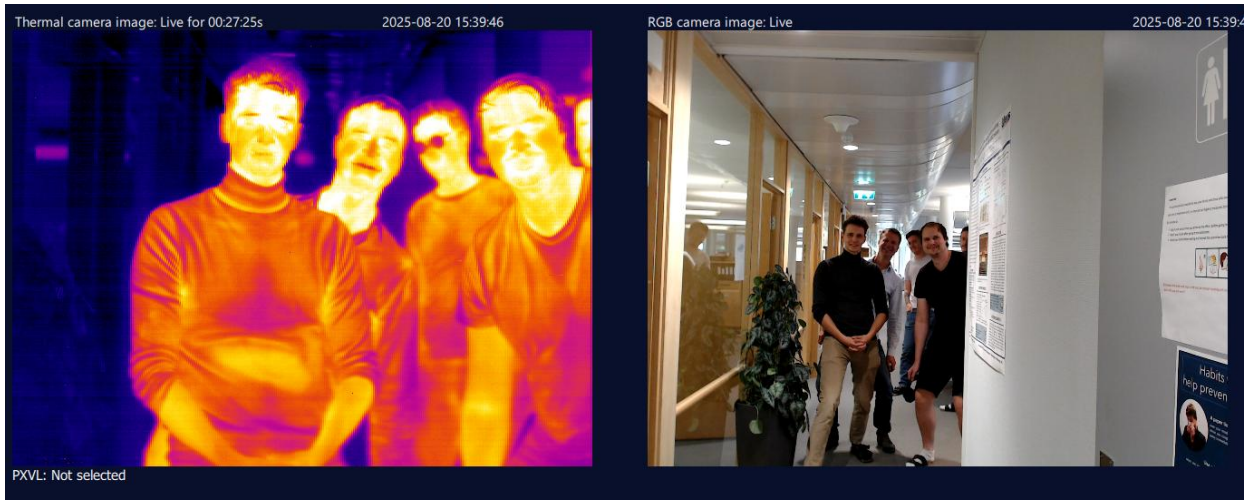
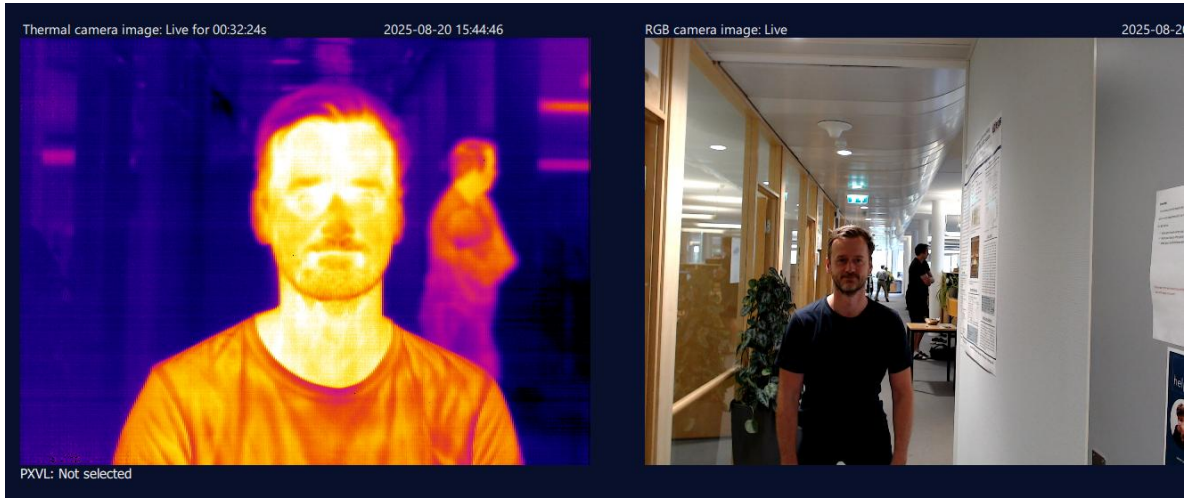


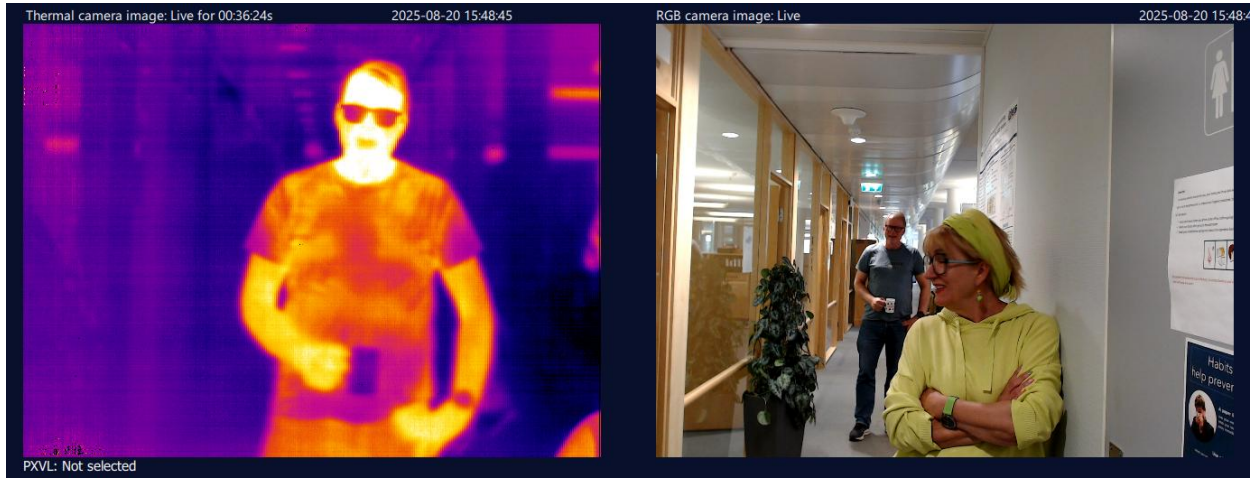


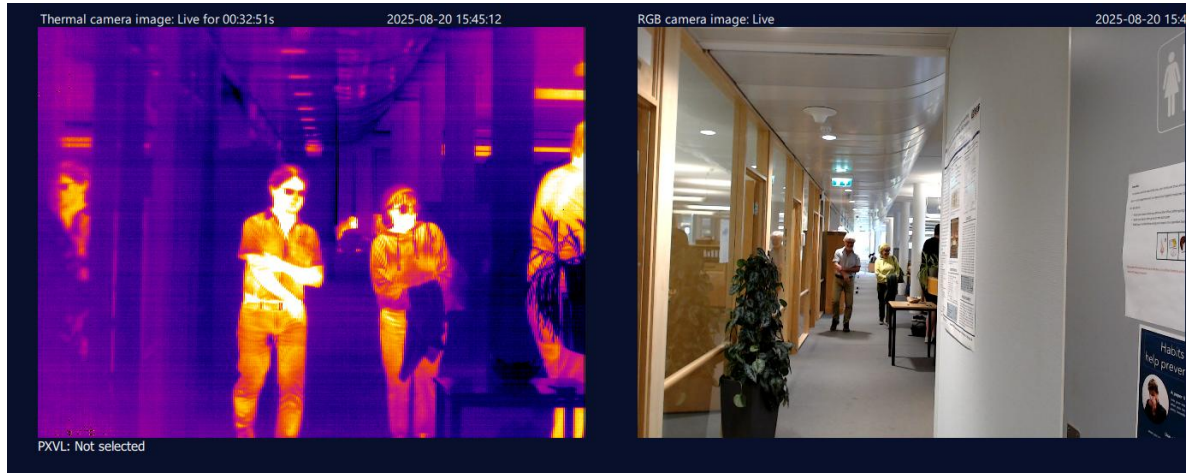
Figure 10: Self-heating issue that affect the output and lead to saturation

Example images after calibration

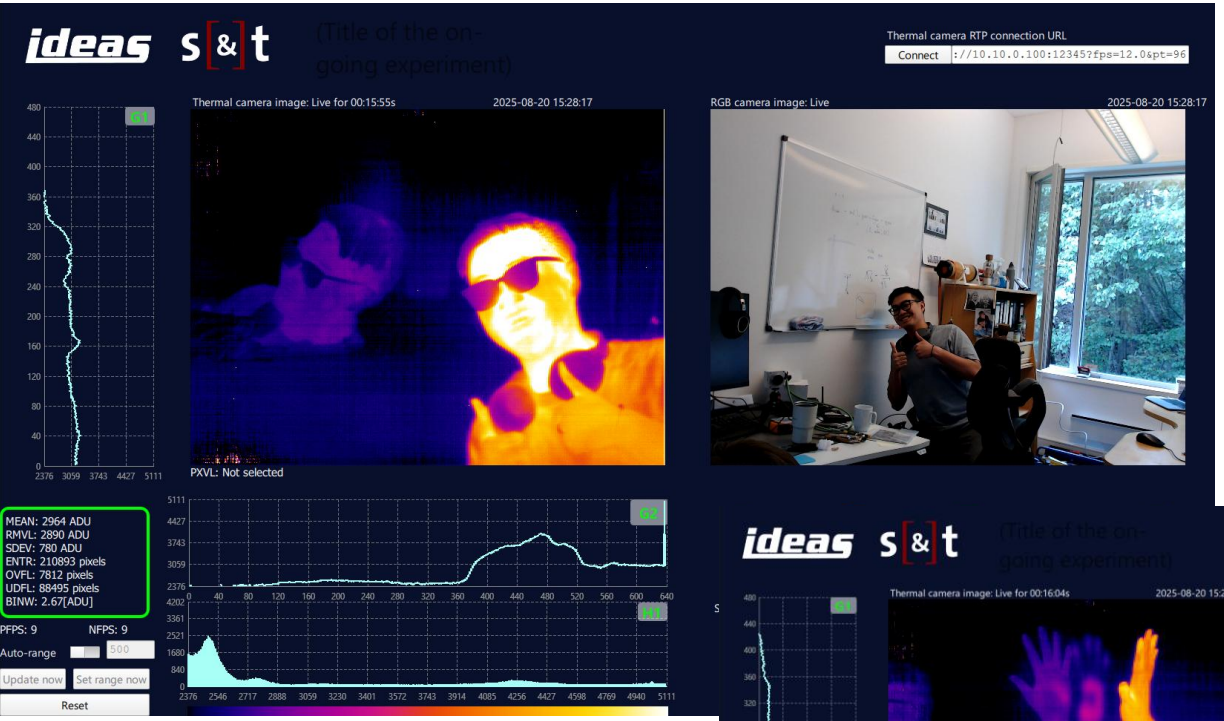


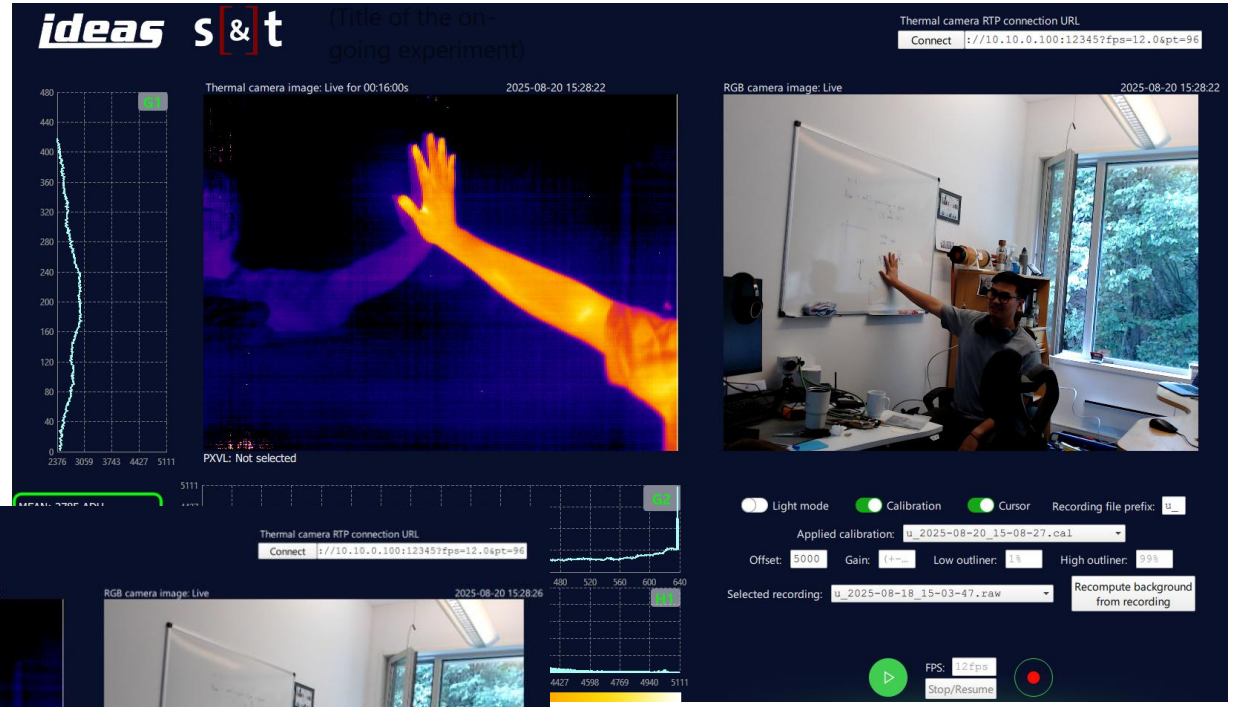


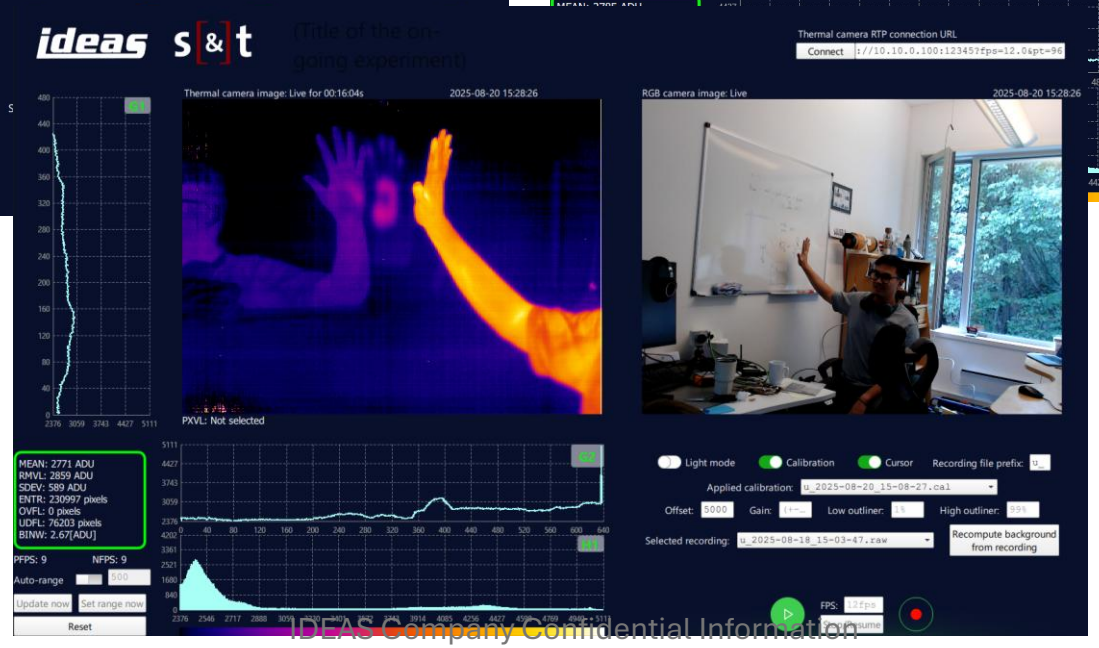




Example images after calibration







IDEAS Company Confidential Information

Improve performance with SIMD

~12 FPS

Feature-complete version, before performance work

~6 FPS

Boost.Histogram + Boost.Accumulators, under a 40 FPS synthetic input

45 FPS

SSE4.1 rewrite: processed rate equals received rate in loopback test

What changed in the per-pixel loop (307,200 pixels per frame)

1. **Vectorised calibration**

   Four pixels per 128-bit register: widen 16 to 32 bit, convert to float, subtract offset, apply gain, rescale; row and column sums in the same pass.

2. **Scalar histogram binning**

   Binning is a data-dependent scatter with no efficient SSE4.1 form, so it stays scalar and uses a bit shift into 1024 bins.

3. **Structure-of-arrays data**

   Offsets and gains from the .cal file stored as contiguous arrays, so each vector needs one aligned load instead of four scattered reads.

4. **Hot-loop clean-up**

   Fixed-size stack arrays instead of containers grown element by element, calibration inlined, running sums instead of accumulator objects.

Thank you!

